

Unit-5

Text processing

String Matching : Given a text $T[0 \dots n-1]$ of ' n ' characters and pattern $P[0 \dots m-1]$ of ' m ' characters. we have to search for occurrence of P in T by making shifts.

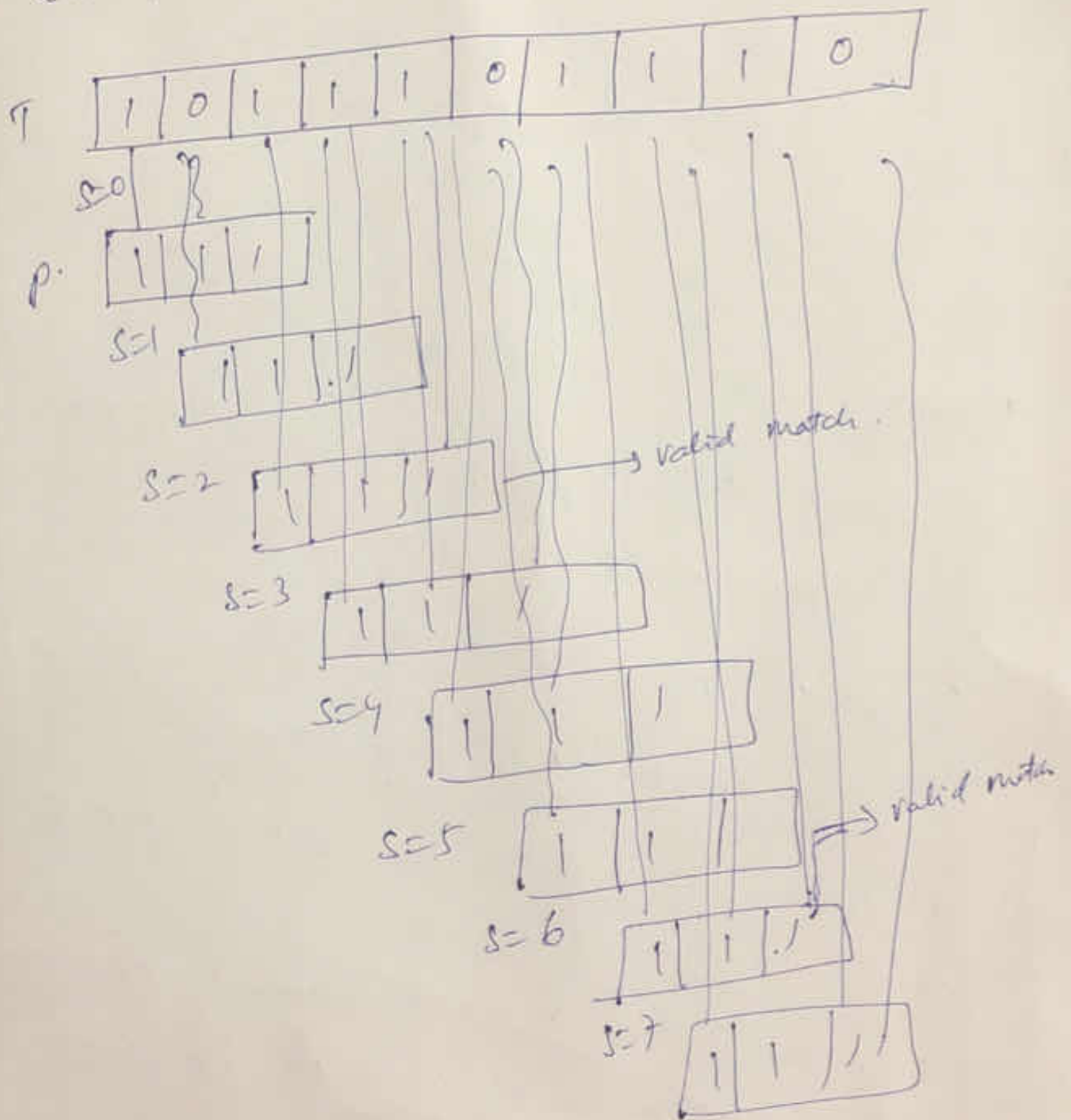
Algorithms for string matching

- ① Naive string matcher / Brute force Algorithm
- ② Boyer Moore Algorithm
- ③ Knuth Morris Pratt Algorithm

Naive string matcher / Brute force Algorithm

- ① It checks all possible placement of pattern in the given text T .
- ② Comparisons are done in forward direction.
- ③ If any mismatch of character, the shift one step right.
- ④ Max. number of shifts are $n-m$.

Question: find all occurrences of $P=111$ in the
text $T=1011101110$.



Brute force string matching Algorithm

①

① naive string matcher.

ex: $T = ABACADABRAC$

$P = ABRA$

$T : A B A C A D A B R A C$

$P : A B R A$

$s=0$

$s=1$

$s=2$

$s=3$

$s=4$

$s=5$

match found.

$s=6$ A B R A.

Algorithm Brute force (T,P)

{

$n = \text{length}(T);$

$m = \text{length}(P);$

for $s=0$ to $n-m$ do

if $(P[0..m-1] == T[s+0 \dots s+m-1])$

print "Pattern occur at shift", s

}

C - Program

```
#include <string.h>
int main()
```

```
{
```

```
char text[10], pattern[5];
```

```
int n, m, s, for n
```

```
printf("Enter text and pattern\n");
```

```
scanf("%s %s", text, pattern);
```

```
int function (char text[], char pattern[])
```

```
{ n = strlen(text);
```

```
m = strlen(pattern);
```

```
return char
```

```
for (s = 0; s <= n - m; s++)
```

```
{ j = 0;
```

```
while (j < m && text[s+j] == pattern[j])
```

```
{ j++; }
```

```
if (j == m)
```

```
return s; }
```

```
return -1;
```

```
}
```

~~0~~

```
int n = strlen(text);
```

```
if (n == -1)
```

```
printf("Not found\n");
```

```
else
```

```
printf("Found at %d",
```

```
s);
```

```
return 0; }
```


2nd day
 Show the comparisons the Brute-force string matcher makes for the pattern $p = 0001$ in the text $T = 000010001010001$ (2)

Time complexity:

$$T(n) = O((n - m + 1)m)$$

$$= O(nm)$$

$T(n) = O(mn)$

$$i = 8 + 6 - \min(5, 1+4)$$

$$i = 14 + 6 - \min(5, 1+4)$$

$$= 20 - 5 = 15$$

Boyer Moore String Matcher	a	b	c	d
last occ	4	5	3	-1

text = a b a c a a b a d c a b a c a b a a b b

pattern = a b a c {a b}

$i = i + m - \min(j, 1 + \text{last}(T(i)))$

$j = m - 1$

- Backward approach - 2 methods
- 1) Bad character heuristic
 - 2) Good suffix heuristic

a b a c a b

a b a c a b

a b a c a b

$$i = 5 + 6 - \min(5, 1+4) = 6$$

$$i = 12 - \min(3, 1+4) = 10 - 3 = 7$$

$$i = 7 + 6 - \min(5, 1+4) = 13 - 5 = 8$$

$$i = i + m - \min(j, 1 + \text{last}(T(i)))$$

$$j = m - 1$$

a b a c a b

a b a c a b

Preprocessing needed to construct B comparison match found

last occurrence table.

Question: Consider a DNA sequence text "AGCTTAGCTTAAGCTTAAGCTTA" and pattern "GCTA". Use Boyer Moore algorithm to find all occurrences of the pattern in the text.

Pseudo code algorithm:

Algorithm Boyer-Moore (T, P, ϵ)
 $\{$ $m = \text{length}(P); n = \text{length}(T);$

$L = \text{LastOccurrence}(P, \epsilon)$

$i = m-1; j = m-1;$
 repeat

if ($T[i] == P[j]$) then

if ($j == 0$) then

return i ;

else $\{$
 $j = j-1;$
 $i = i-1;$
 $\}$

else $\{$

$i = i + m - \min(j, L[T[i]])$ ~~last occurrence~~

$j = m-1;$

$\}$

until ($i > n-1$)

return -1; $\}$

Time Complexity

$O(mn)$ -
 worst case

Knuth Morris Pratt (KMP) String Matching

Prefix function for given pattern (π)

$P[i]$	0	1	2	3	4	5	6
	a	b	a	b	a	c	a
$\pi[i]$	0	0	1	2	3	0	1

Q2:

a	a	b	a
0	1	0	1

Q3:

A	B	A	B	C	A	B	A	B
0	0	1	2	0	1	2	3	4

Q4:

$P[i]$	1	2	2	1	1	2
$\pi[i]$	0	0	0	1	1	2

Q5: Compute the prefix function π for the pattern
ababbabbabbabbabbabb

$P[i]$	a	b	a	b	b	a	b	b	a	b	b	a	b	a	b	b	a	b	b
$\pi[i]$	0	0	1	2	0	1	2	0	1	2	0	1	2	3	4	5	6	7	8

① prefix function must be calculated for pattern.
Length of longest prefix repeated.

② π - LPS array
Longest prefix suffix array

③ Time comp.
 $O(n+m)$.

where n - length of text
 m - length of pattern.

$i=0, j=0$; KMP string matched

while ($i \leq n$)

{

if ($T[i] == P[j]$) then

{ $i++$;

$j++$;

}

if ($j == m$)

{ Print match occur at ^{index} ~~pos~~ $i-j$;

$j = \pi[j-1]$;

}

}

else

{ if ($i \neq 0$) then

$j = \pi[j-1]$

else

$i++$;

}

}

a b a b a b a b c a
0 0 1 2 3 4 5 6 0 1

(2)

Q: using Knuth Morris Pratt algorithm find whether the pattern $p = 122112$ is in the text 21221221121121

1212 or not

Sol:

Prefix Function

i	1	2	3	4	5
$p[i]$	1	2	2	1	2
$\pi[i]$	0	0	0	1	2

Text = 2 1 2 2 1 2 2 1 1 2 1 1 2 1 2
P = 1 2 2 1 1 2

Diagram showing alignment of pattern P with text T:

```

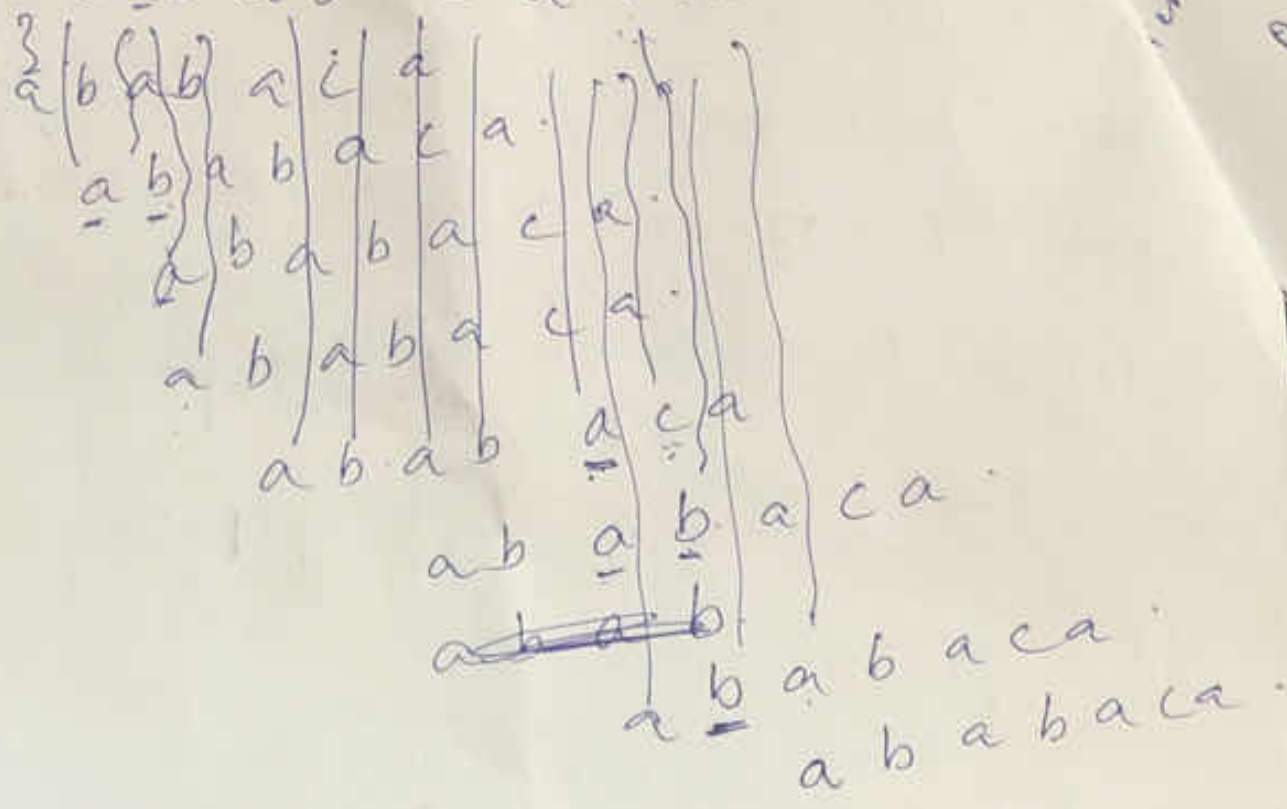
      0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
      ↓
Text = 2 1 2 2 1 2 2 1 1 2 1 1 2 1 2
      P = 1 2 2 1 1 2
           1 2 2 1 1 2
              1 2 2 1 1 2
                 1 2 2 1 1 2
                    1 2 2 1 1 2
                       1 2 2 1 1 2
                          1 2 2 1 1 2
                             1 2 2 1 1 2
                                1 2 2 1 1 2
                                   1 2 2 1 1 2
  
```

Comp = 11 e

match found at
index = 10 - 7
= 3

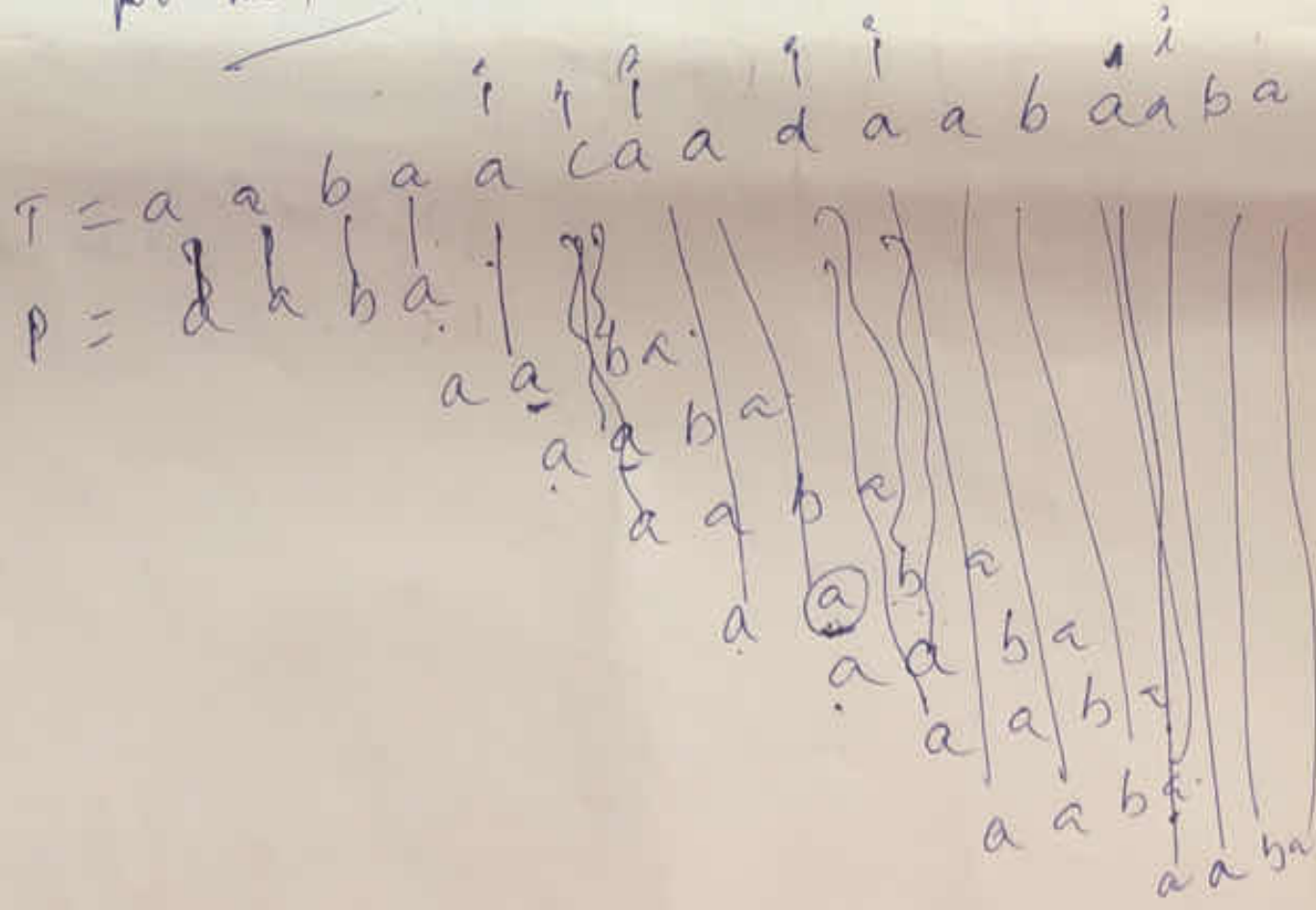
1 2 2 1 1 2 stop
1 2 2 1 1 2 stop

Text = $\begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 \\ b & a & c & b & a & b & a & b & a & c & b & c & b & a & b \end{matrix}$



no match

Q3:



ans

Text = A B A B D A B A S D A B A B C A B A B

P = A B A B C A B A B
0 1 2 3 4 5 6 7 8

A B A B C A B A B

A B A B C A B A B
A B A B C A B A B
A B A B C A B A B
A B A B C A B A B

0	1	2	3	4	5	6	7	8
A	B	A	B	C	A	B	A	B
0	0	1	2	0	1	2	3	4

$J = T[i-1]$
 $J = 2$
 $j = 1$

A B A B C A B A B

19-9
= 10

P =

0	1	2	3	4	5
1	2	2	1	1	2
0	0	0	1	1	2

$i = 4 - 6 = 5$

$J =$

2 1 2 2 1 2 2 1 2 1 2 2

2 2 2 1 1 2 2 2 2 2 2 2

1 2 2 1 1 2 2 2 2 2 2 2

1 2 2 1 1 2 2 2 2 2 2 2

Comparisons = 12

① Compute prefix function for pattern.

$P = ababaca$

i	0	1	2	3	4	5	6
$P[i]$	a	b	a	b	a	c	a
$\pi[i]$	0	0	1	2	3	0	1

②

pattern	=	$\pi[i]$	0	1	2	3
			a	a	b	a
		$\pi[i]$	0	1	0	1

$T =$ a a b a a c a a d a a b a a b a
 $P =$ a a b a a c a a d a a b a a b a
 a a b a
 a a b a

1. అక్షరాలు, పదాలు, పదాల పదాలు, పదాల పదాలు
 2. పదాల పదాలు, పదాల పదాలు, పదాల పదాలు

Standard Trie

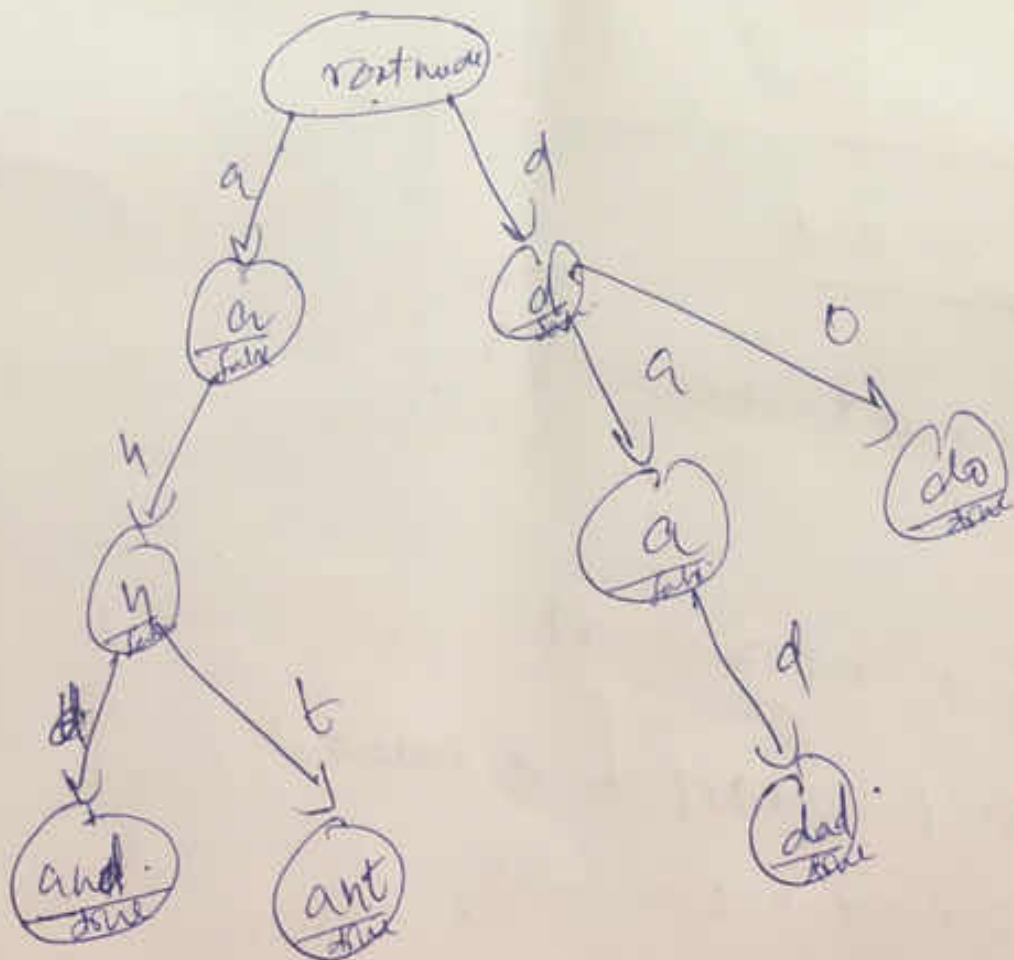
trie - tree like data structure which is used to store dynamic set of strings.

* It is commonly used for efficient retrieval and storage of keys in large data set.

* operation $\leftarrow$ Insertion
 Search
 Deletion

Standard Trie

Example: and, ant, dad, do



Representation of node of a trie

child[] - add of child nodes.

↳ max. limit of child[] is 26.

isEndofWord
isWordEnd

- boolean field - If word ends
it has true else it will have false

struct trienode

{
struct trienode * child[26];

~~isEndofWord~~
bool isEndofWord;

}

for (i=0; i<26; i++)

{

ptr->child[i] = NULL;

}

return ptr;

}

Create a node

struct trienode * create_node()

{

ptr = (struct trienode *)

malloc(sizeof(struct trienode));

ptr->isEndofWord = false;

insert : Key = and.

$$Index = 'a' - 'a' = 0$$

$$Index = 'n' - 'a' = 13$$

$$Index = 'd' - 'a' = 3$$

void TrieInsert(struct tnode
*root, char *key)

```
{ int Index;
  cur = root;
  while(*key != '\0') {
    Index = *key - 'a';
    cur
```

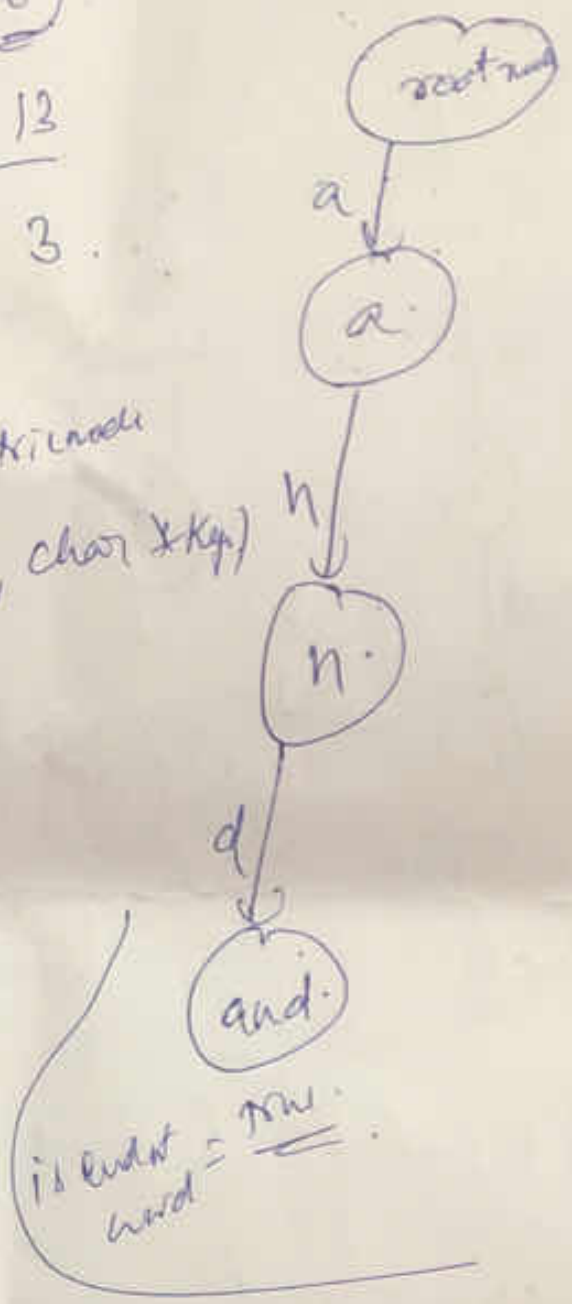
```
if (cur->child[Index] == NULL)
```

```
{
  cur->child[Index] = createNode();
```

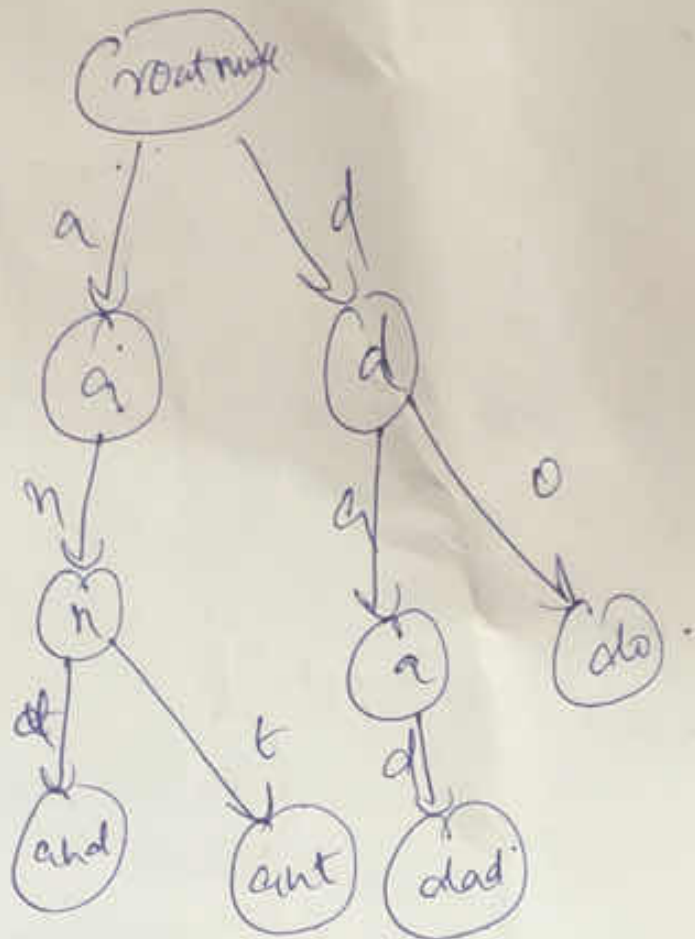
```
cur = cur->child[Index];
```

```
key++;
```

```
} cur->isEndWord = true; }
```



Search



Search for Key = "and".

bool Trie-Search (struct trienode *root, char *key)

{

 cur = root;

 while (*key) {

 index = *key - 'a';

 if (cur->child[index] == null)

 return false;

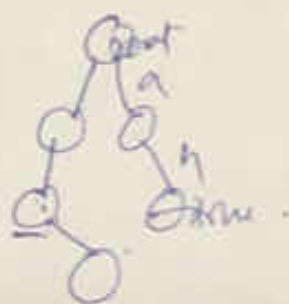
 cur = cur->child[index]; key++; }

return (cur != NULL && cur->isEndofWord);

?

trie delete

```
struct node * trie-delete(struct node *root,  
                           char *key, int d=0)  
{  
    int len = strlen(key);  
    if (root == NULL)  
        return root;
```



```
    if (len == d)
```

```
    {
```

```
        if (root->isEndofWord == true)
```

```
            root->isEndofWord = false;
```

```
        if (isEmpty(root))
```

```
        {
```

```
            free(root);
```

```
            root = NULL;
```

```
        }
```

```
    return root;
```

```
}
```

```
int index = *key - 'a';
```

```
root->child[index] = trie-delete(root->child[index], key key,  
                                d+1);
```

```
if (isEmpty(root) && root->isEndofWord == false)
```

```
{ free(root); root = NULL; }
```

```
return root;  
}
```

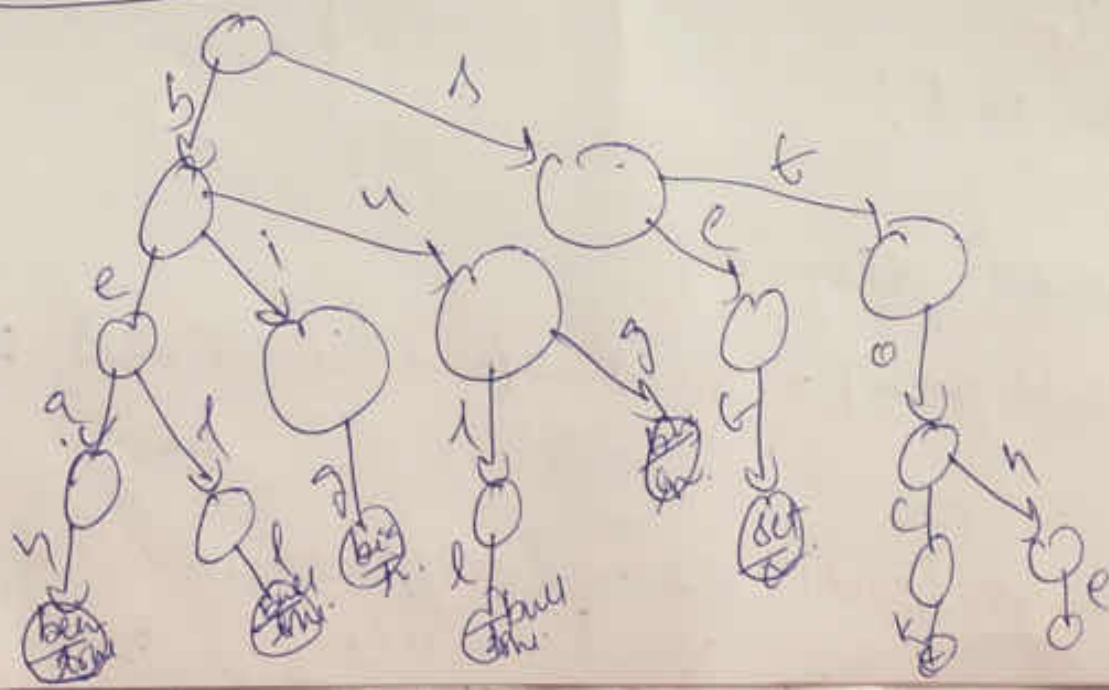
Self, drawing, abstract, set, stone, set, set
to set, set, set, set

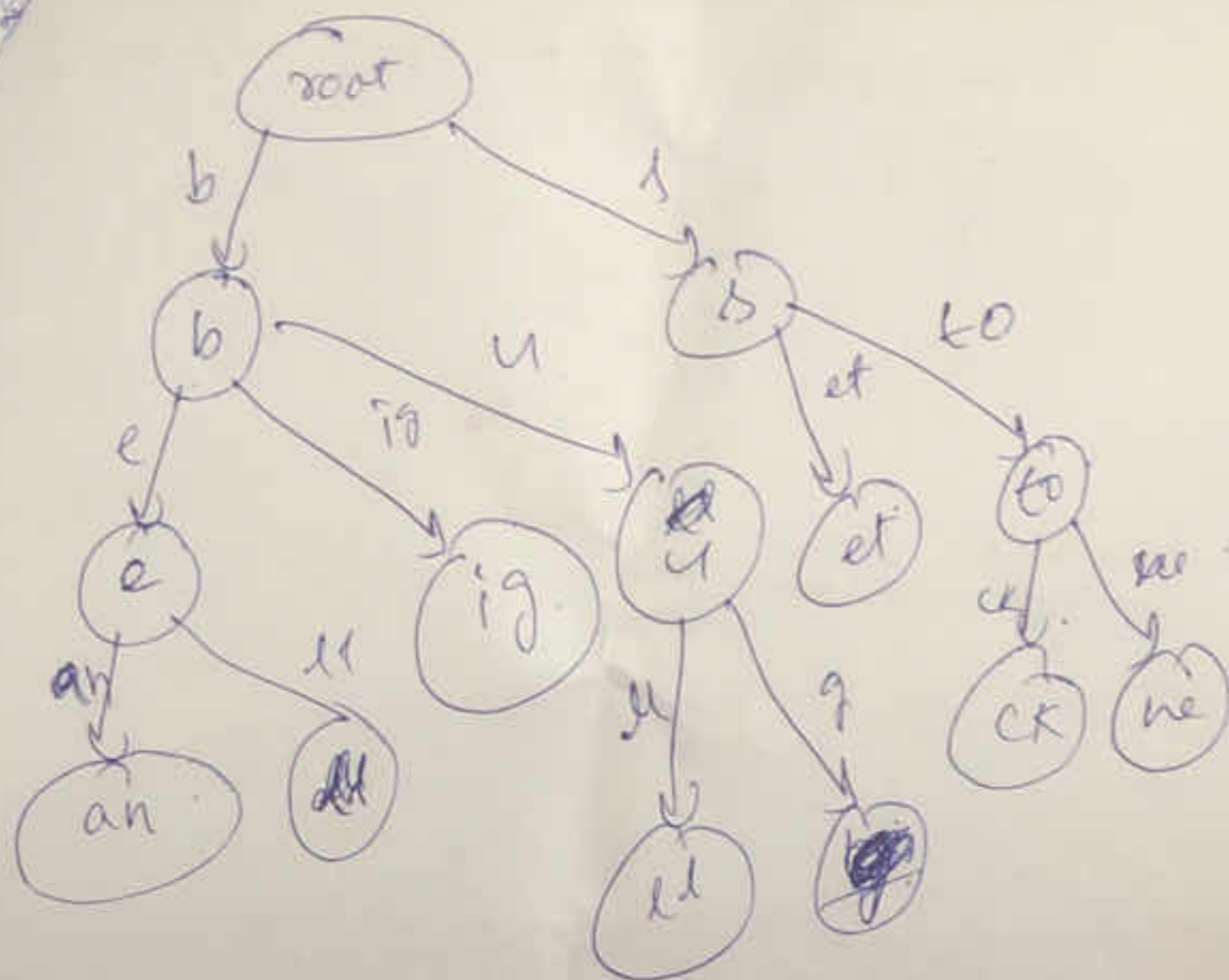
Compressed Trie (patricia trie) / Radix trie

Definition: It is an optimized version of standard trie with single children are merged, using edges labeled with substrings instead of individual characters. Hence it reduces the space usage and improves efficiency for storing and searching strings with common prefixes.

Q: Construct a compressed trie for the following set of strings {bean, bell, big, bull, bug, set, stock, stone}

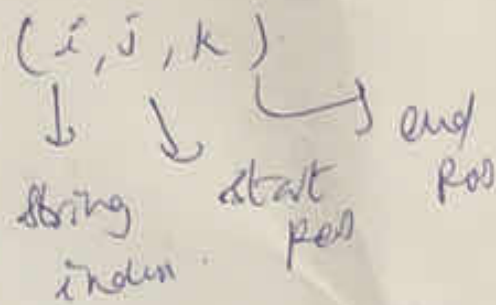
① Construct standard trie



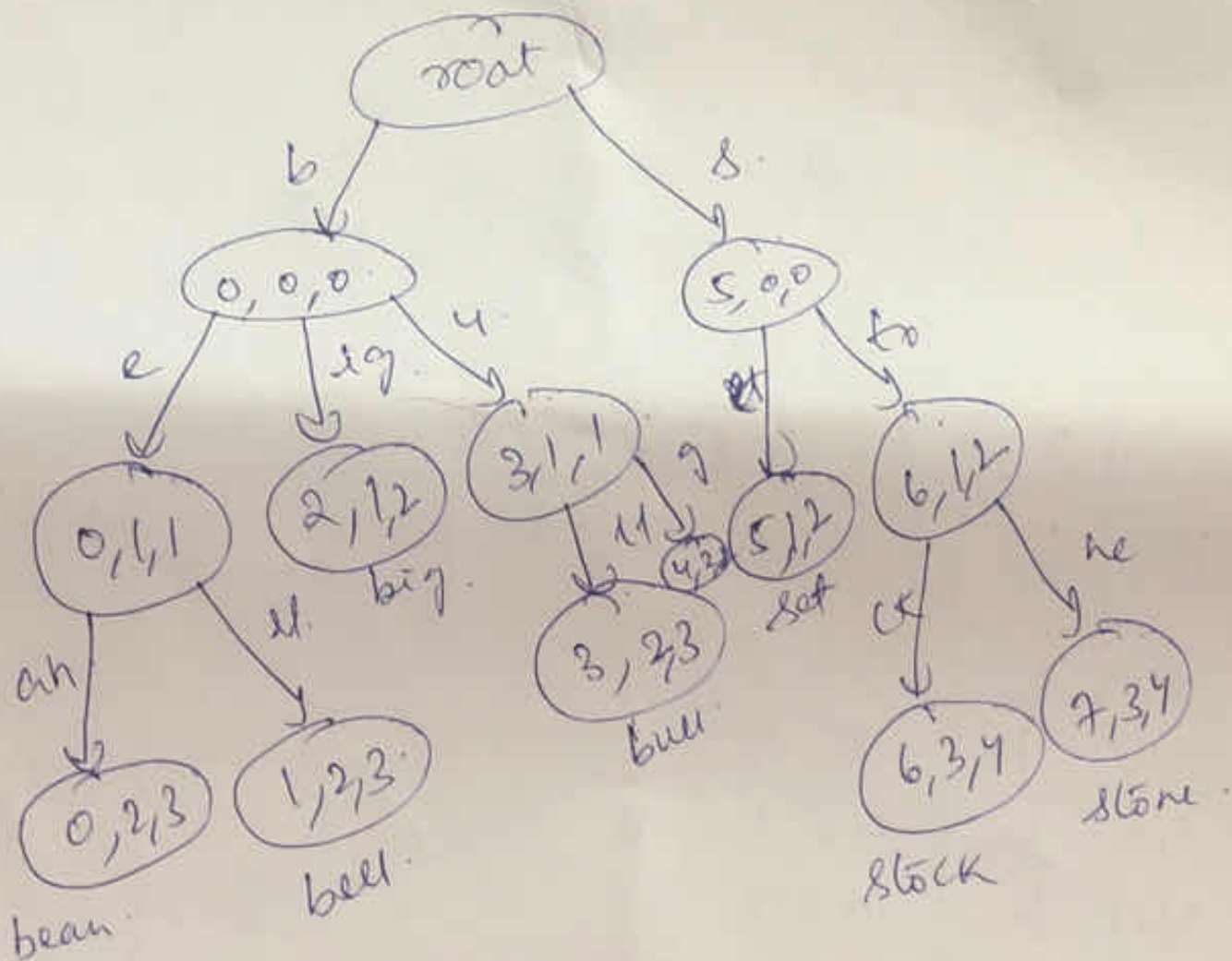


	0	1	2	3	4	
$s[0]$	b	e	a	n		(0,0,0)
$s[1]$	b	e	l	l		
$s[2]$	b	i	g			
$s[3]$	b	u	l	l		
$s[4]$	b	u	g			
$s[5]$	s	e	t			
$s[6]$	s	t	o	c	k	
$s[7]$	s	t	o	n	e	

triple representation → compact representation



Construct compressed tree



Q: Construct compressed tree for { bear, bell, bed, bull, buy, sell, stock, stop }.

Prefix tree

construction of tree for substrings of a given string.

* Use - To identify all substrings.

example: google.

substrings set = { e, le, gle, ogle, google }

consider in alphabetical order.

